

Composição

A outra metade da orientação a objetos: a relação "tem um", que não se resolve com herança.

Leia isto antes dos exercícios.

Cada seção termina com perguntas. Responda elas antes de passar para a seguinte.

Sumário

01	O problema: a poção que virou personagem	2
02	"É um" e "tem um"	4
03	Composição é só um atributo	5
04	Delegação: cada um cuida do que é seu	6
05	Uma coleção com dono	7
06	Polimorfismo outra vez, agora fora do combate	8
07	Como o objeto se apresenta: <code>__str__</code>	10
08	A porta de leitura: <code>@property</code>	11
09	Erros comuns ao compor	12
10	Herança ou composição: como decidir	13
11	Estudo de caso: o inventário completo	14
12	Perguntas finais	16
13	Sequência de estudo	17

Na Parte B você aprendeu herança: uma classe **é um** tipo de outra. Com ela o seu RPG saiu de uma classe só para uma família de classes.

Agora vem o outro lado. Nem toda relação entre dois objetos é "é um". A maioria, na verdade, é "**tem um**". E usar herança onde cabia "tem um" é o erro mais comum de quem acabou de aprender herança.

Esta parte tem uma ideia central só: **composição**. Um objeto guarda outro objeto dentro de si.

SEÇÃO 1

O problema: a poção que virou personagem

O seu RPG vai ganhar itens. Poção, bomba, espada. Você acabou de aprender herança, então a primeira ideia é natural:

```
class Pocao(Personagem):  
    def __init__(self):  
        super().__init__("Poção de vida", 1, 0, 0, 0)
```

Isso **funciona**. O Python não reclama. E é aí que mora o problema.

Repare no que essa poção ganhou de graça:

```
pocao = Pocao()  
pocao.atacar(goblin)      # uma pocao atacando  
pocao.defender()          # uma pocao se defendendo  
pocao.esta_vivo()         # uma pocao viva  
pocao.receber_dano(10)    # uma pocao levando dano
```

Uma poção que ataca. Uma poção que está viva. Uma poção com defesa e com mochila.

Nada disso faz sentido, e nada disso vai dar erro na tela. O programa roda. Ele só está errado por dentro, e você vai descobrir daqui a duas semanas, quando alguém chamar `pocao.atacar()` por engano e o jogo fizer uma coisa absurda em silêncio.

A PERGUNTA QUE RESOLVE

"Poção **é um** Personagem?" Diga a frase em voz alta. Se ela soar estranha, herança está errada.

PERGUNTAS

1. Depois de `class Pocao(Personagem)`, liste três métodos que a poção passou a ter e que não fazem sentido nenhum para ela.
2. O código desta seção roda sem erro nenhum. Explique por que isso é pior do que ter dado erro.
3. Diga a frase "é um" para cada par e responda se ela é verdadeira: Poção e Personagem; Mago e Personagem; Bomba e Item.
4. Um colega diz que herdar de `Personagem` foi bom porque "assim a poção já vem com o nome pronto". Responda a ele.

SEÇÃO 2

"É um" e "tem um"

Existem duas relações diferentes entre classes, e elas se dizem com duas frases diferentes.

"É um" é herança. A filha é um tipo da mãe.

- Guerreiro **é um** Personagem. Verdade.
- Mago **é um** Personagem. Verdade.
- Dragão **é um** Personagem. Verdade.

"Tem um" é composição. Um objeto guarda outro.

- Personagem **tem uma** mochila. Verdade.
- Mochila **tem** itens. Verdade.
- Poção **é um** Personagem? Falso. Personagem **tem** poções? Verdade.

A poção não é um tipo de personagem. Ela é uma coisa que o personagem **carrega**. A relação é de posse, não de tipo.

Guarde esta tabela mental:

- Se você quer **outros comportamentos para a mesma coisa**, use herança.
- Se você quer **juntar coisas diferentes**, use composição.

O Guerreiro e o Mago são a mesma coisa (personagens) agindo diferente: herança. O personagem e a mochila são coisas diferentes que andam juntas: composição.

PERGUNTAS

1. Escreva a frase certa, "é um" ou "tem um", para cada par: Dragão e Personagem; Personagem e mochila; Bomba e Item; mochila e itens.
2. Um Escudo tem nome, tem defesa e pode quebrar. Ele é um Personagem? Ele é um Item? Justifique as duas respostas.
3. Herança ou composição. Diga qual é o caso de cada par: um Carro e um Motor; um Carro e um Veículo; um Aluno e uma Turma; um Aluno e uma Pessoa.
4. Explique, com suas palavras, por que "aproveitar código pronto" não é motivo suficiente para herdar.

SEÇÃO 3

Composição é só um atributo

A parte técnica é decepcionante de tão simples. Composição não tem sintaxe nova. É um atributo que guarda um objeto.

Você já fez isso sem saber o nome:

```
self.nome = nome          # o atributo guarda um texto
self.vida = vida          # o atributo guarda um numero
```

Agora:

```
self.inventario = Inventario() # o atributo guarda um OBJETO
```

Só isso. `self.inventario` é um atributo como qualquer outro. A diferença é que o valor dentro dele não é um número nem um texto: é outro objeto, com os métodos dele.

E por isso você pode escrever:

```
jogador.inventario.adicionar(pocao)
```

Leia da esquerda para a direita: pegue o jogador, pegue a mochila dele, mande a mochila guardar a poção.

PERGUNTAS

1. O que fica guardado dentro de `self.inventario` depois de `self.inventario = Inventario()`? Compare com o que fica guardado em `self.nome`.
2. Leia `jogador.inventario.adicionar(pocao)` da esquerda para a direita e descreva em português o que cada ponto faz.
3. O que acontece se você escrever `Inventario` sem os parênteses? Em que momento o erro aparece, e qual é a mensagem?
4. Composição precisa de sintaxe nova? Justifique com o que você viu nesta seção.

SEÇÃO 4

Delegação: cada um cuida do que é seu

Quando um objeto tem outro dentro, aparece uma pergunta nova: **quem responde o quê?**

A resposta é a mesma da Parte A: cada objeto cuida do próprio estado. O personagem não sabe como uma lista de itens funciona. Ele só sabe pedir.

```
class Personagem:
    def __init__(self, nome, vida, ataque, defesa, pocoes):
        ...
        self.inventario = Inventario()
```

O personagem **não** faz isto:

```
self.inventario._itens.append(pocao)    # mexendo na lista dos outros
```

Ele faz isto:

```
self.inventario.adicionar(pocao)        # pedindo
```

Chamar um método do objeto que você guarda, em vez de mexer nos dados dele, tem nome: **delegação**. Você delega o trabalho para quem é dono dele.

O ganho é o mesmo do encapsulamento da Parte 2. Se amanhã a mochila passar a ter peso máximo, essa regra entra dentro do `adicionar`, e nenhum personagem precisa saber que a regra existe.

PERGUNTAS

1. Qual das duas linhas respeita a delegação, e por quê:
`self.inventario._itens.append(x)` ou `self.inventario.adicionar(x)`?
2. A mochila passa a ter peso máximo. Em qual classe e em qual método entra essa regra? Quantas outras classes precisam mudar?
3. Explique por que o `Personagem` não precisa saber que a mochila guarda os itens dentro de uma lista.
4. Cenário novo. Amanhã a mochila passa a guardar os itens num dicionário em vez de lista. Quem quebra, se todo mundo usou os métodos?

SEÇÃO 5

Uma coleção com dono

A mochila é uma classe que existe para cuidar de uma lista. Isso parece pouco, e é justamente o ponto.

```
class Inventario:
    def __init__(self):
        self._itens = []

    def adicionar(self, item):
        self._itens.append(item)

    def quantidade(self):
        return len(self._itens)

    def tirar(self, numero):
        indice = numero - 1
        if indice < 0 or indice >= len(self._itens):
            return None
        return self._itens.pop(indice)
```

Compare com a alternativa, que é guardar uma lista solta no personagem:

```
self.itens = []          # lista crua, aberta para qualquer um
```

Com a lista crua, todo mundo que quiser usar um item precisa saber que ela começa no índice zero, precisa lembrar de checar se o número existe, e precisa lembrar de remover o item depois de usar. Essa regra fica **espalhada** por todo lugar que toca na lista.

Com a classe `Inventario`, a regra fica em um lugar só. O `tirar` devolve `None` quando o número não existe, e ninguém mais precisa se preocupar com isso.

SINAL DE QUE A CLASSE VALE A PENA

ela protege pelo menos uma regra. Se a sua classe só repassa chamadas sem proteger nada, você provavelmente não precisava dela.

PERGUNTAS

1. Cite uma regra concreta que a classe `Inventario` protege e que uma lista solta não protegeria.
2. Por que `tirar` devolve `None` em vez de deixar o programa quebrar? Quem chama esse método?
3. O `listar` numera a partir de 1, mas a lista começa no índice 0. Onde essa conversão acontece, e por que é bom que ela aconteça em um lugar só?
4. Quando uma classe **não** vale a pena? Use o sinal descrito no fim da seção.

SEÇÃO 6

Polimorfismo outra vez, agora fora do combate

Na Parte B o polimorfismo apareceu no combate: `inimigo.atacar(jogador)` fazia coisas diferentes conforme o inimigo.

Agora ele volta, e é a mesma ideia com outras classes.


```

class Item:
    def __init__(self, nome):
        self.nome = nome

    def usar(self, dono, inimigo):
        print(self.nome, "nao faz nada")

class PocaDeVida(Item):
    def __init__(self):
        super().__init__("Poção de vida")
        self.cura = 25

    def usar(self, dono, inimigo):
        dono.curar(self.cura)

class Bomba(Item):
    def __init__(self):
        super().__init__("Bomba")
        self.dano = 30

    def usar(self, dono, inimigo):
        inimigo.receber_dano(self.dano)

```

Repare que os dois `usar` recebem **exatamente os mesmos parâmetros**, e fazem coisas opostas. A poção cura quem usou. A bomba fere o outro. Cada item decide sozinho quem ele afeta.

E no jogo:

```

item = jogador.inventario.tirar(numero)
item.usar(jogador, inimigo)

```

Essas duas linhas não perguntam que item é. Não têm `if`. Funcionam com poção, com bomba, e com qualquer item que você inventar amanhã.

Note também que `Item` usa **herança**, e `Personagem` usa **composição** para guardar o inventário. As duas ferramentas convivem no mesmo programa. Elas não competem: cada uma responde uma pergunta diferente.

PERGUNTAS

1. Os dois `usar` têm exatamente os mesmos parâmetros. Por que isso é obrigatório para o polimorfismo funcionar?
2. Você cria um `Elixir`, que cura o dono e fere o inimigo na mesma chamada. Quantas linhas do menu precisam mudar? Justifique.
3. Nesta seção, `Item` usa herança e `Personagem` usa composição. Explique por que as duas aparecem no mesmo programa sem competir.
4. Ligue o conceito ao nome. Diga qual pilar cada trecho representa, herança, composição, polimorfismo ou encapsulamento: - `class Bomba(Item):` - `self.inventario = Inventario()` - `item.usar(jogador, inimigo)` fazendo coisas diferentes por item - `self._itens` com underscore

SEÇÃO 7

Como o objeto se apresenta: `__str__`

Para listar a mochila na tela, você precisa transformar um item em texto. A tentação é guardar isso de fora:

```
print(numero, "-", item.nome)
```

Funciona, mas espalha de novo: todo lugar que mostra um item precisa lembrar de usar `.nome`.

Python tem um método especial para isso, e ele se chama `__str__`. Ele responde a pergunta "como você vira texto?".

```
class Item:
    def __str__(self):
        return self.nome
```

Com ele escrito, o `print` passa a funcionar direto no objeto:

```
print(pocao)          # Poção de vida
print(numero, "-", pocao)
```

`__str__` é da mesma família do `__init__`: um método com dois underscores de cada lado, que o Python chama sozinho na hora certa. Você nunca escreve `item.__str__()`. Você escreve `print(item)`, e o Python chama por você.

No `Personagem` ele também cabe:

```
def __str__(self):
    return f"{self.nome} ({self._vida}/{self._vida_maxima})"
```

E aí `print(jogador)` mostra `Thoric (120/120)`.

PERGUNTAS

1. Por que o `__str__` usa `return` e não `print`? O que acontece se você trocar um pelo outro?
2. Quem chama o `__str__`? Cite outro método que também roda sozinho, sem você escrever a chamada.
3. Sem o `__str__`, o que aparece na tela quando você imprime um item? O que essa saída está te dizendo?
4. Depois de escrever o `__str__` do `Personagem`, em quantos lugares do jogo o texto `Thoric (120/120)` passa a funcionar de graça?

SEÇÃO 8

A porta de leitura: `@property`

Na Parte 2 você trocou `self.vida` por `self._vida` e fechou a porta. Isso resolveu o problema do `inimigo.vida = -999`, mas cobrou um preço: agora nem **ler** a vida dá mais.

```
print(goblin.vida)    # AttributeError
```

Ler nunca foi o problema. O problema era escrever. Python tem uma forma de liberar um sem liberar o outro:

```
class Personagem:
    @property
    def vida(self):
        return self._vida
```

O `@property` faz um método parecer um atributo. Com ele:

```
print(goblin.vida)      # 40, funciona
goblin.vida = -999      # AttributeError: property has no setter
```

Leitura liberada, escrita continua proibida. E repare no detalhe importante: **quem usa não muda nada**. Continua escrevendo `goblin.vida`, sem parênteses, como se fosse um atributo comum. O objeto trocou o interior sem quebrar ninguém de fora, que é a mesma promessa da Parte 2.

É por isso que Python não tem `getVida()` e `setVida()` espalhados como outras linguagens. Você começa com um atributo simples e, no dia em que precisar de uma regra, transforma em `@property` sem mexer em quem usa.

PERGUNTAS

1. Depois do `@property`, qual das duas linhas funciona e qual quebra: `print(g.vida)` ou `g.vida = 50`?
2. A Parte 2 fechou a leitura junto com a escrita. Explique por que isso foi um exagero.
3. Quem usa a classe precisa mudar alguma coisa quando `vida` vira uma property? Justifique.
4. Por que em Python você não precisa escrever `getVida()` e `setVida()` desde o começo, como em outras linguagens?

SEÇÃO 9

Erros comuns ao compor

1. Herdar o que devia ser guardado. `class Inventario(Personagem)`. A mochila ganha vida, ataque e defesa. Diga a frase: "mochila é um personagem". Não é.

2. Mexer na lista do outro. `jogador.inventario._itens.append(x)`. Se você escreveu o underscore de fora da classe, você furou o encapsulamento. Use o método.

3. Criar classe que não protege nada. Uma classe `Item` que só tem `nome` e nada mais poderia ser um texto. Ela passa a valer a pena quando ganha o `usar`, que é diferente em cada tipo.

4. Esquecer que o objeto guardado precisa nascer. Se você declara `self.inventario` mas nunca escreve `Inventario()`, o atributo fica valendo `None` e a primeira chamada quebra com `AttributeError: 'NoneType' object has no attribute 'adicionar'`.

5. Empilhar delegação. `jogo.jogador.inventario.itens.primeiro.nome` é sinal de que alguém está alcançando longe demais. Quanto mais pontos seguidos, mais frágil.

PERGUNTAS

1. Dos cinco erros da seção, qual deles não dá mensagem nenhuma na tela? Por que ele é o mais perigoso?
2. Olhe a linha `jogo.jogador.inventario.itens.primeiro.nome`. Explique o que há de errado nela sem falar de sintaxe.
3. Você declara `self.inventario` mas esquece o `Inventario()`. Qual é a mensagem, e em que momento ela aparece?
4. A partir de que ponto uma classe `Item`, que no começo só guardava um nome, passa a valer a pena?

SEÇÃO 10

Herança ou composição: como decidir

Faça as perguntas nesta ordem.

1. A frase "X é um Y" é verdadeira? Se não for, pare. Não é herança.

2. X e Y são a mesma coisa fazendo coisas diferentes? Guerreiro e Mago são personagens agindo diferente: herança. Personagem e mochila são coisas diferentes: composição.

3. Você quer herdar só para reaproveitar um método? Isso é armadilha. Reaproveitar código não é motivo suficiente para herdar. Se a frase "é um" não for verdadeira, guarde o objeto e chame o método dele.

Na dúvida, **prefira composição**. Ela é mais fácil de desfazer. Trocar um objeto guardado por outro é uma linha; desmontar uma hierarquia de herança que ficou errada é uma tarde.

PERGUNTAS

1. Aplique as três perguntas da seção a este caso: uma classe `Arma` e uma classe `Espada`. Qual é a relação?
2. Agora a este: uma classe `Personagem` e uma classe `Arma`. Qual é a relação?
3. A seção manda preferir composição na dúvida. Responda por quê, falando do custo de desfazer cada uma.
4. Um colega quer que `Inventario` herde de `list`, para ganhar o `append` de graça. Responda a ele usando a regra do "é um".

SEÇÃO 11

Estudo de caso: o inventário completo

Juntando tudo, com o RPG da Parte 2 como base.

```

class Item:
    def __init__(self, nome):
        self.nome = nome

    def usar(self, dono, inimigo):
        print(self.nome, "nao faz nada")

    def __str__(self):
        return self.nome

class PocaodeVida(Item):
    def __init__(self):
        super().__init__("Poção de vida")
        self.cura = 25

    def usar(self, dono, inimigo):
        dono.curar(self.cura)
        print(dono.nome, "bebeu a poção e recuperou", self.cura, "de vida")

class Bomba(Item):
    def __init__(self):
        super().__init__("Bomba")
        self.dano = 30

    def usar(self, dono, inimigo):
        inimigo.receber_dano(self.dano)
        print("A bomba explodiu em", inimigo.nome)

class Inventario:
    def __init__(self):
        self._itens = []

    def adicionar(self, item):
        self._itens.append(item)

    def esta_vazio(self):
        return len(self._itens) == 0

    def quantidade(self):
        return len(self._itens)

    def listar(self):
        for numero, item in enumerate(self._itens, 1):
            print(numero, "-", item)

    def tirar(self, numero):
        indice = numero - 1
        if indice < 0 or indice >= len(self._itens):
            return None
        return self._itens.pop(indice)

```

E no `Personagem`, a composição em duas linhas:

```
self.inventario = Inventario()
for _ in range(pocoas):
    self.inventario.adicionar(PocaoDeVida())
```

Repare no que **não** aconteceu: nenhuma das suas classes filhas mudou. Nem Guerreiro, nem Mago, nem Goblin, nem o Dragão. Todas continuam chamando `super().__init__(nome, vida, ataque, defesa, pocoas)` do mesmo jeito. O número de poções, que antes virava um contador, agora vira itens na mochila, e isso é assunto interno do `Personagem`.

Esse é o mesmo resultado da Parte 1, agora do outro lado: você trocou o interior de uma classe e as filhas não ficaram sabendo.

PERGUNTAS

1. `Personagem` guarda um `Inventario`, e `Inventario` guarda `Item`. Escreva as duas frases "tem um" que descrevem isso.
2. `PocaoDeVida` herda de `Item`. Escreva a frase "é um" que justifica.
3. O `usar` da poção chama `dono.curar()` em vez de mexer em `dono._vida`. Qual conceito da Parte 2 isso respeita?
4. Se a mochila passasse a ter limite de 5 itens, em qual arquivo e em qual método entraria essa regra? Quantas outras classes precisariam mudar?

FECHAMENTO

Perguntas finais

1. Em uma frase cada, qual é a diferença entre herança e composição?
2. Um `Escudo` deve herdar de `Item` ou ser guardado por ele? Justifique com a regra do "é um".
3. O `Inventario` poderia ser só uma lista dentro do `Personagem`. Dê um motivo concreto para ele ser uma classe.
4. Por que `item.usar(jogador, inimigo)` não precisa de nenhum `if` perguntando o tipo do item?
5. `__str__` e `@property` resolvem problemas diferentes. Qual é cada um?
6. Releia o erro 1 da Seção 9. Por que ele é perigoso mesmo sem dar erro na tela?

FECHAMENTO

Sequência de estudo

1. Leia a Seção 1 até o fim antes de ver a solução. Sinta o incômodo da poção que ataca.
2. Diga em voz alta as frases "é um" e "tem um" para cada par de classes do seu jogo. Anote quais são verdadeiras.
3. Escreva a classe `Item` com `nome`, `usar` e `__str__`. Teste com `print`.
4. Escreva `PocaoDeVida` e `Bomba`. Confirme que o `usar` das duas tem a mesma assinatura e efeitos opostos.
5. Escreva a classe `Inventario`. Teste `adicionar`, `listar` e `tirar`, inclusive com um número que não existe.
6. Ligue o inventário ao `Personagem`. Confirme que nenhuma classe filha mudou.
7. Acrescente `__str__` ao `Personagem` e `@property` na vida. Confirme que ler voltou a funcionar e escrever continua bloqueado.
8. Só depois disso, apague o `usar_pocao`. Ele não tem mais função.